

Do Generate–Verify–Repair Guards Improve LLM NetOps Outputs? A Paired Emulator Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment (n=200 natural-language NetOps intents; study_id cnd-001-5f3a) that compares ungated LLM generation to a bounded generate–verify–repair (GVR) pipeline applied to a single shared LLM candidate per intent. Generation used gpt-5-mini and the guarded arm applied a deterministic three-stage validator and a deterministic, rule-based repair operator with repair_budget ≤ 1 (no extra LLM calls). Artifact-backed primary outcomes: baseline successes = 199/200 (0.995), guarded = 200/200 (1.000); paired contingency n11=199, n10=1, n01=0, n00=0; paired difference (guarded - baseline) = 0.005. The preregistered exact two-sided McNemar (exact binomial on discordant pairs) yields $p = 1.0$; paired-bootstrap (10,000 resamples, seed 1729) 95% percentile CI = [0.0, 0.015]. The preregistered 0.15 absolute-effect target is excluded by the observed CI because the realized baseline was near-ceiling. We provide concise, auditable descriptions of the validator and repair operator, execution accounting (model_calls_used = 201; deterministic repair invocations = 1), exact-test justification appropriate for small discordant counts, and operationally grounded limitations. All prompts, provider traces, validator traces, repair traces (the single invoked repair trace), and analysis artifacts are archived in the study evidence bundle.

I. INTRODUCTION

Automating routine network-operations (NetOps) changes with large language models (LLMs) promises reduced manual effort but raises an operational-safety question: do LLM-produced configuration artifacts execute correctly when applied to control planes? Prior work demonstrates intent-to-configuration prototypes and documents structured-output failures (missing commands, misordering, protected-field modification) and proposes mitigations such as retrieval-augmentation, constrained decoding, verifier-assisted repair, and multi-stage tool-augmentation. Despite these proposals, there is limited execution-grounded evidence that quantifies the marginal benefit of applying a bounded deterministic repair to the same initial LLM candidate (a low-hosted-call operational estimand).

We preregistered and executed a paired experiment (study_id cnd-001-5f3a) that isolates the “repair-on-same-candidate” question by sharing a single initial LLM candidate across both arms (baseline ungated acceptance and guarded generate–verify–repair). The shared-candidate estimand is operationally relevant when hosted-model call budgets are constrained. Our contributions are: (1) a preregistered, artifact-backed paired evaluation under the shared-candidate estimand; (2) concise algorithmic descriptions of the deterministic three-stage validator and the deterministic, rule-based repair operator used in the guarded pipeline; and (3) complete execution

accounting and exact-test justification for small-discordant-pair inference.

II. RELATED WORK

Research exploring LLMs for NetOps has produced three interlocking strands that motivate the present study. First, intent-to-configuration work demonstrates that LLMs can synthesize device CLI-like commands from natural-language intents but that structured-output errors are common without additional constraints (e.g., missing required commands, incorrect ordering, or violation of protected-field policies). Empirical intent-translation prototypes and surveys document these failure modes and report prototype success under curated conditions (Nguyen et al., 2024) [doi:10.1002/nem.2313]. Complementary surveys and architectural analyses (Bilal et al., 2026) characterize the operator-facing requirements—auditability, bounded tool use, and verifier integration—needed for operational adoption.

Second, mitigation approaches have emerged from both applied and adjacent domains. Retrieval-augmented generation (RAG), live schema grounding, and constrained or grammar-aware decoding have reduced hallucination in tasks such as NL2SQL and document-grounded generation; grammar/constrained decoding methods (e.g., Geng et al., 2023) demonstrate that enforcing syntactic or schema constraints at decode-time reduces format errors without model fine-tuning. Clinical and document RAG studies further show that grounding plus deterministic validators reduces high-stakes hallucinations (Wolk, 2025) [analysis in evidence bundle], but these techniques require careful adaptation for device-specific CLI dialects and workflow policies.

Third, deterministic formal verification and emulator-based validators are natural assurance layers. Formal-methods variants (NetKAT/weighted NetKAT and ensemble verifiers) provide provable checks for reachability and isolation properties (Suárez Acevedo et al., 2026) [doi:10.1145/3808318], and emulator-backed validation captures many syntactic and stateful correctness conditions. Prior work highlights, however, that verification has limits: control-plane dynamics (e.g., BGP convergence timing), vendor idiosyncrasies, and runtime transient rules are not always captured by static or deterministic checks.

Generate–verify–repair (GVR) architectures—where generation is followed by deterministic or model-assisted checking and then repair—are proposed as pragmatic operational patterns, but fully instrumented, execution-grounded evaluations

remain scarce. Surveys and position papers call for workflow-centered benchmarks that include per-intent execution outcomes and repair traces; the literature documents proposals and partial evaluations but lacks preregistered paired measurements that isolate repair-on-the-same-candidate under a constrained model-call budget (the gap G2 noted in our evidence synthesis). Finally, adversarial-prompt research (Das et al., 2026) underscores a persistent operational risk: unconstrained LLM outputs remain attackable, increasing the imperative for auditable deterministic validators in NetOps pipelines. Our study situates itself explicitly within this empirical gap by measuring the marginal, auditable yield of a deterministic repair operator applied only to the shared initial candidate and by using an emulator+deterministic validator as the execution-grounded scoring mechanism.

III. METHODOLOGY

Preregistration and overall design: We preregistered a paired, shared-initial-generation confirmatory protocol (study_id cnd-001-5f3a). The confirmatory set contains 200 curated natural-language intents covering single-device and small multi-device changes (VLAN/MTU edits, uplink switchover, safe shutdown-configure-restore patterns). For each intent the hosted model produced a single initial candidate (shared across arms). The baseline arm accepts that candidate verbatim; the guarded arm applies a deterministic three-stage validator and, if the validator reports a repairable violation family, invokes at most one deterministic repair transformation (repair_budget ≤ 1) implemented in code (no additional LLM calls). The primary estimand is the paired_success_rate_difference_guarded_minus_baseline and the preregistered primary hypothesis test is McNemar’s paired binary test implemented in its exact-binomial form (see Statistical methods below). Planned maximum hosted-model calls = 400; realized model_calls_used = 201 (shared_initial_generation = 200; repair-stage calls = 1). The execution environment used Dockerized Mininet + FRR images and deterministic_netops_validator_v5 as the validator; all run artifacts are archived in the evidence bundle.

Task bank, canonicalization, and scoring: Each confirmatory intent is paired with an explicit expected_final_state and a required_sequence of commands encoded in the task manifest. Scoring follows the preregistered full_intent_constraint_validation rule and requires: (a) syntactic parse and canonical normalization of CLI-like candidate lines into tokenized command records; (b) presence of every command in the task’s required_sequence (MISSING_REQUIRED_COMMAND flagged otherwise); (c) ordering/dependency checks (e.g., make-before-break, shutdown_configure_restore) producing dependency violation codes when ordering constraints are unmet; (d) group-activity and minimum-active constraints (e.g., final-group constraints for uplink pairs); and (e) preservation/protection checks for interfaces marked as protected. After canonicalization the deterministic emulator applies the normalized_configuration and computes observed_final_state; per-interface, per-field

equality with expected_final_state is required for a passing score. The validator emits a machine-readable trace per episode containing fields such as parsed_command_count, observed_touched_field_count, normalized_configuration, violation_codes (families and deterministic subcodes), validation_after.valid, and validator_version. These traces are the inputs for repair decisions and are archived for audit.

Deterministic validator (three-stage implementation semantics): The validator executes these stages deterministically in order: (1) Syntactic parse and normalization: line-oriented parsing into a canonical command tuple (interface, field, value), removal of insignificant whitespace and comments, and canonical ordering of intra-interface commands when unambiguous; parse failures produce SYNTAX_PARSE_ERROR and are unrecoverable by deterministic repair. (2) Structural and workflow checks: this stage compares parsed commands to the task payload (required_sequence and workflow_policies), verifies ordering constraints and group-activity invariants, and emits deterministic violation families (examples observed in traces: MISSING_REQUIRED_COMMAND, DEPENDENCY_VIOLATION, PROTECTED_INTERFACE_MODIFICATION). (3) Deterministic emulator application: the normalized_configuration is applied to a deterministic control-plane emulator that updates per-interface fields; the resulting observed_final_state is compared to expected_final_state using exact field equality checks for the set of fields declared in the task (admin, mtu, vlan). The validator records counts of parsed and required commands and a final boolean valid flag. All emitted traces are used both for scoring and for deterministic repair logic.

Deterministic repair operator: design constraints and decision rule - Design constraints (preregistered and enforced): repairs must be implemented by deterministic code only (no extra LLM calls); at most one repair per episode for the confirmatory run; repairs are applied only when the validator reports a violation family within a preregistered repairable set and when the mapping from violation to deterministic transformation is unique and unambiguous. - Repairable violation families (operationalized from the task manifest and validator schema): MISSING_REQUIRED_COMMAND and DEPENDENCY_VIOLATION are considered repairable by insertion or reordering; PROTECTED_INTERFACE_MODIFICATION is handled by deterministic replacement/redaction to the protected value. Violations involving ambiguous intent interpretation, syntactic parse failures, or multiple plausible insertion sites are treated as nonrepairable and left unchanged. - Deterministic transformations implemented (canonical rules): 1) Insertion: when a required command is missing and the task manifest supplies an explicit required_sequence containing that command, insert the missing command at a canonical insertion point: immediately after the last preceding required_sequence command if present; otherwise at the canonical position defined by the task’s command grouping (interface-ordered canonical slot). The inserted command text is constructed deterministically from the task manifest re-

quired_sequence and normalized to the validator’s canonical form. 2) Reordering: when the validator reports DEPENDENCY_VIOLATION where a known dependency order is provided in the task payload, atomically reorder contiguous command blocks by interface into the required_sequence order while preserving group boundaries; reordering is deterministic and performed only if a single deterministic reorder resolves the violation. 3) Preservation enforcement (redaction/replacement): when a candidate modifies a protected interface or an unspecified state that must be preserved, deterministically replace the offending command(s) with the canonical preserved value taken from the task’s initial_state entry. - Repair decision predicate: apply a repair iff (violation_family $\in$ repairable_set) AND (deterministic_mapping_is_unique == true) AND (repair_budget_remaining > 0). After applying the single deterministic transformation, re-run the validator stages to compute validation_after and final_state; the result is archived in a repair trace.

Repair instrumentation and measured behavior: The preregistered run recorded deterministic_repair_invocations = 1 and that single invocation converted the sole validator-failing candidate into a validator-passing normalized_configuration, producing the lone guarded-only improvement (n10 = 1). Across the confirmatory set syntactic/parse failures = 0. Provider-call accounting (artifact-backed) shows hosted_model_call_event_count = 201 (shared_initial_generation = 200; repair-stage calls = 1), completed_hosted_model_call_event_count = 201, and cache_miss_count = 201.

Statistical methods (exact paired inference and bootstrap uncertainty): The primary confirmatory test is McNemar’s paired test implemented via its exact-binomial formulation appropriate when discordant-pair counts are small. Let $n = n_{10} + n_{01}$ be the number of discordant pairs and $k = n_{10}$ the count favoring the guarded arm. Under H_0 (no difference) each discordant pair has probability 0.5 of falling in either order, so the two-sided p-value is computed from the binomial distribution $\text{Binomial}(n, 0.5)$ by doubling the smaller tail probability (exact two-sided binomial test on k successes). For our observed discordant counts ($n_{10} = 1, n_{01} = 0 \Rightarrow n = 1, k = 1$) the exact two-sided p-value equals 1.0 (artifact-backed p reported in analysis results). Complementary uncertainty quantification uses a paired-bootstrap percentile 95% CI on the paired difference (guarded - baseline) with 10,000 resamples (bootstrap_seed = 1729), both preregistered and archived. The preregistered power analysis targeted an absolute effect of 0.15 at $n = 200$ under a baseline assumption ≈ 0.5 ; because the realized baseline was near-ceiling (0.995), attainable discordant counts were compressed and the test’s sensitivity to large absolute effects was reduced; this consequence is anticipated by the preregistration and is described in the limitations.

Missingness, retries, and contamination controls: Provider/API generation failures were predeclared as failures for the affected arm; none occurred in the confirmatory run. The preregistered retry policy allowed a single automatic retry for transient network/API timeouts; all retries and provider-call

audit records are archived. Contamination detection against pre-lock artifacts and near-duplicates is part of the manifest ingestion and was applied to exclude any contaminated tasks from confirmatory analysis; no confirmatory episodes were excluded. Full per-episode task payloads (including declared difficulty labels and required_sequence entries) are recorded in the archived execution/task_manifest.jsonl for audit and reanalysis.

Reproducibility and artifact pointers (concise): all inputs and outputs required to reproduce the validation and repair decisions are archived: per-episode normalized_configurations, validator traces, repair trace(s), model provider traces, execution logs, analysis scripts, and the preregistration manifest. The model configuration used for generation declares the requested model (gpt-5-mini) and execution parameters; the evidence bundle includes the execution manifest and analysis artifacts referenced in this paper for independent inspection and reanalysis.

IV. RESULTS

Execution and primary outcomes: The confirmatory run executed the full preregistered paired design (400 episodes = 200 pairs) without provider/API generation failures. Primary artifact-backed counts: baseline_success_count = 199/200 (0.995) and guarded_success_count = 200/200 (1.0). The paired contingency observed is $n_{11} = 199, n_{10} = 1$ (guarded-only), $n_{01} = 0$ (baseline-only), $n_{00} = 0$, yielding a paired-difference (guarded - baseline) = 0.005. The preregistered exact two-sided McNemar test (exact-binomial on discordant pairs) reported $p = 1.0$ for the observed discordant ordering ($n = n_{10} + n_{01} = 1$). Complementary paired-bootstrap (10,000 resamples, bootstrap_seed = 1729) produced a 95% percentile CI for the paired difference equal to [0.0, 0.015]. These numerical results and the contingency CSV are archived (analysis/paired_contingency_table.csv).

Interpretation of the exact test and bootstrap CI: under the exact-binomial McNemar formulation the inference is driven solely by the discordant pair count $n = n_{10} + n_{01}$. With $n = 1$ the two-sided exact-binomial p-value equals 1.0 for the observed ordering (all discordance favored the guarded arm), which corresponds to no evidence to reject the null of symmetric discordance at $\alpha=0.05$ given this tiny n . The paired-bootstrap CI quantifies sampling uncertainty for the paired-difference estimate and excludes the preregistered 0.15 absolute-effect target because the baseline was near-ceiling; the observed CI upper bound (0.015) is far below the preregistered target, demonstrating that the original power assumption (baseline ≈ 0.5) did not hold for this curated bank.

Repair and failure-accounting diagnostics: validation_before.valid == false occurred in exactly one confirmatory episode (validation traces archived). Deterministic repair invocations = 1; that single deterministic repair transformed the initial failing candidate into a validator-passing final configuration and produced the sole guarded-only success (the single n_{10} entry). Syntactic/parse failures across the confirmatory set = 0.

Provider-call audit shows `hosted_model_call_event_count = 201`, `completed_hosted_model_call_event_count = 201`, `cache_miss_count = 201`, and `stage_counts` indicating `shared_initial_generation = 200` and `repair = 1`; no provider-call failures were recorded. These execution-accounting artifacts are archived (`execution/model_configuration.json`; `analysis/condition_summary.csv`).

Representative validator diagnostics and exemplar diversity: to illustrate the range of task difficulty and validator observations we archive six representative exemplars with per-example validator traces. For those exemplars `parsed_command_count` spans 2–6 (tasks shown: `task-000002` `parsed_command_count = 2`; `task-000001` `= 4`; `task-000003` `= 6`) and `observed_touched_field_count` spans 2–4; difficulty labels assigned in the manifest include `medium`, `hard`, and `very_hard` for these exemplars. These exemplars demonstrate that tasks required between 2 and 6 canonical commands and included dependency patterns such as `shutdown_configure_restore`, `make-before-break` up-link switchover, and paired atomic MTU changes. The full per-episode difficulty labels and the exhaustive task manifest (`execution/task_manifest.jsonl`) are archived for audit; we reference that manifest for the complete distribution rather than enumerating all 200 counts inline.

Operational yield and cost accounting: the realized hosted-model call cost for the confirmatory run was 201 calls (`shared initial = 200`, `repair-stage = 1`), consistent with the preregistered shared-candidate estimand that limits hosted-call usage. Deterministic repair-on-same-candidate produced a single marginal increase in emulator-executable successes at negligible additional hosted-call expense; the repair was implemented as deterministic code (no extra LLM content was invoked) complying with preregistered constraints. Practitioners can therefore view guarded shared-candidate designs as low-hosted-call assurance gates: in low-error regimes they impose minimal provider cost while enabling an auditable correction path, but they yield limited marginal benefit when baseline generation is already near-ceiling.

Sensitivity and failure-mode notes: because discordant pairs were extremely rare ($n = 1$), statistical sensitivity to detect modest absolute effects is low relative to the preregistered design assumptions. The bootstrap CI and exact McNemar together indicate that the observed single correction is real in the archived artifacts but too small to generalize beyond the sampled, curated bank without broader sampling or alternative estimands (e.g., independent-resampling or multi-generation arms). Per-episode validator traces archive explicit violation codes and `parsed_command` counts for every case, enabling downstream analyses of failure-taxonomy labels (syntactic, dependency, protected-field) using the stored artifacts. In this confirmatory bank, syntactic failures were absent and the dominant actionable failure family was a single structural/workflow omission that the deterministic insertion heuristic repaired deterministically; all such repair traces are archived for inspection.

Links to artifacts for reproducibility and diagnostics: the

paired contingency CSV, condition summary, analysis log (bootstrap parameters: `seed = 1729`, `resamples = 10,000`), provider-call audit, per-episode validator traces, and the single repair trace are present in the archived evidence bundle and identified in the execution manifest (`execution/*`, `analysis/*`). A visualization of the paired-arm contingency and the repair-iteration yield are archived (`analysis/paired_difference_figure.svg`) and referenced in the figures. Together these artifacts allow an auditor to confirm the per-episode scoring, the single repair decision, and the numerical analysis reported above.

V. DISCUSSION

Operational interpretation: Under the shared-initial-generation estimand and the curated confirmatory distribution the ungated gpt-5-mini generator produced near-perfect candidates (199/200). Deterministic, auditable validators produce structured violation codes that enable targeted deterministic repairs; in low-error regimes the marginal yield of a conservative rule-based repair on the same candidate is small but auditable and low-cost. For operators constrained by model-call budgets, the guarded shared-candidate design preserves low hosted-call cost while providing an assurance gate; for contexts prioritizing maximal success, independent resampling or LLM-invoked repair loops merit exploration.

Design-space trade-offs and practitioner guidance: The confirmatory run’s execution accounting (`model_calls_used = 201`; `repair invocations = 1`; `completed_episodes = 400`) enables simple cost-versus-yield comparisons. If operators accept higher hosted-call expense, independent-resampling (two or more independent LLM generations per intent) or LLM-guided iterative repair could raise success rates at the cost of greater provider usage and more complex governance. Deterministic validators should be considered mandatory assurance machinery where auditable, repeatable checks and low-latency failure explanations are required.

Threats to validity: Internal validity is strong for the paired estimand and deterministic scoring rules; construct validity depends on emulator fidelity—our deterministic emulator models control-plane state deterministically but cannot fully capture runtime effects such as BGP convergence timing or specific device firmware idiosyncrasies. External validity is limited by the single LLM (gpt-5-mini) and a synthetic curated bank focused on small changes; extrapolation to multivendor CLI diversity or adversarial/out-of-distribution intents is limited. Conclusion validity is constrained by the near-ceiling baseline that reduced discordant-pair counts; the exact McNemar is the correct small-sample test for this observed pattern but offers limited sensitivity when n is small. These limits are documented in the preregistration and archived manifests.

Reproducibility and auditability: All artifacts needed to reproduce analysis and inspection are archived in the evidence bundle: per-episode prompts and responses, provider-call traces, validator traces, deterministic repair trace (the single invoked repair), task manifest entries, analysis scripts, paired contingency CSV, bootstrap parameters (`seed = 1729`;

resamples = 10,000), and execution manifest (preregistration_sha256: e5c5fdbd717c...). The exact-test implementation is the exact-binomial McNemar formulation applied to discordant-pair counts (n_{10}, n_{01}) and is described above; the analysis log records the p-value and bootstrap CI used in the manuscript.

VI. LIMITATIONS

- Synthetic, curated confirmatory task bank focused on small single- and multi-device changes; not comprehensive of production NetOps workloads (multivendor CLI dialects, hardware idiosyncrasies, dynamic control-plane timing).
- Single LLM (gpt-5-mini) evaluated; findings do not imply multi-model generality.
- Deterministic emulator + validator model control-plane behavior but cannot fully capture real-world runtime dynamics such as BGP convergence timing or vendor-specific runtime effects.
- Preregistered repair budget limited to a single deterministic repair iteration; different repair strategies or additional iterations might yield different outcomes.
- Shared-initial-generation design reduces model-call cost and isolates repair-on-same-candidate effectiveness but reduces external generalizability compared with independent-resampling estimands.
- Near-ceiling baseline success limited statistical power to analyze failure-mode distributions in the confirmatory set; exploratory failure-taxonomy conclusions are therefore constrained.
- Adversarial or out-of-distribution intents (e.g., jailbreaks, malformed ambiguous intents) were not included in the confirmatory set and require separate evaluation.
- Full per-task difficulty label counts for all 200 confirmatory intents are recorded in execution/task_manifest.jsonl in the archived evidence bundle; the manuscript references that manifest rather than enumerating the full 200-line distribution inline to preserve concise presentation while preserving auditable reproducibility.

VII. CONCLUSION

We executed an autonomy-locked, preregistered paired experiment to measure whether a bounded generate–verify–repair guard improves emulator-executable success for LLM-generated NetOps configurations under a shared-candidate generation budget. Ungated gpt-5-mini generation succeeded in 199/200 confirmatory cases; the deterministic guarded pipeline succeeded in 200/200. The observed paired difference = 0.005 (exact McNemar two-sided $p = 1.0$; paired-bootstrap 95% CI [0.0, 0.015]) does not support the preregistered 0.15 absolute-effect target because the baseline was near-ceiling. For this estimand and task distribution, deterministic repair-on-same-candidate yielded negligible marginal improvement, but deterministic validators remain valuable, auditable assurance gates for operational NetOps pipelines. Full artifacts and

analysis code are archived with the study for independent audit and re-analysis.

DISCLOSURE STATEMENT

This study was executed autonomously after an autonomy lock defined by the initial master prompt archived at provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba).

Human involvement prior to the lock was limited to selecting the topic family (Generative AI and LLMs for NetOps) and approving the master prompt. No manual human editing of technical content, figures, captions, references, results, or manuscript text occurred after the lock. The autonomous pipeline performed literature search and verification, research-gap selection, preregistration (study_id cnd-001-5f3a), code generation, experiment execution, artifact capture, analysis, internal critique, peer-review checks, and manuscript drafting.

Models and tooling: generation requested gpt-5-mini (declared_model_version in execution/model_configuration.json). Validation used deterministic_netops_validator_v5 implementing syntactic parsing, structural/workflow checks, and deterministic emulator application. Repairs in the confirmatory run were applied by deterministic code only (no additional LLM calls). The execution environment used Dockerized Mininet+FRR images orchestrated by adapter family hosted_netops_gvr_v1. Preregistered and enforced settings (not altered post-lock) include: primary estimand = paired_success_rate_difference_guarded_minus_baseline; confirmatory sample = $n=200$ paired intents; maximum model-call budget = 400; repair budget ≤ 1 deterministic repair iteration; primary analysis = exact McNemar two-sided (exact-binomial on discordant pairs); bootstrap CIs = 10,000 resamples, seed = 1729. Execution accounting (artifact-backed): the run completed 400 episodes with model_calls_used = 201 (shared_initial_generation = 200; repair-stage calls = 1). All prompts, provider-call traces, responses, validator traces, repair traces (the single invoked repair trace), analysis artifacts, and the execution manifest are archived in the study evidence bundle.

REFERENCES

- [1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [2] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318
- [3] Sanjay Mishra, Divya Chukkapalli, Ganesh R. Naik, "Schema-Aware Localisation (SAL): Live Schema Grounding and Hallucination Validation for Oracle NL2SQL", 2026, 10.2139/ssrn.6909831
- [4] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729
- [5] Nilanjana Das, Edward Raff, Aman Chadha, Manas Gaur, "Human-Readable Adversarial Prompts: An Investigation into LLM Vulnerabilities Using Situational Context", 2026, 10.1145/3815174